UNIVERSITY OF THE PEOPLE

CS 2301-01 - AY2027-T1

Unit 5 Assignment

Submitted by: Mubashir Ali

Instructor: Joseph Ego

Date: October 01, 2026

Command-Line Automation and Disk Scheduling in a Banking Environment

Introduction

A bank's IT infrastructure must perform routine administrative tasks reliably while processing a large volume of transactions. Command-line scripting can automate repetitive operations such as daily backups, log-file management, and disk-usage monitoring. Bash is particularly appropriate for this purpose because it provides variables, conditional statements, loops, functions, and the ability to combine operating-system utilities into automated workflows (GNU Project, 2025). At the same time, efficient disk scheduling is important when a workload involves traditional hard disk drives (HDDs), where excessive head movement can increase access time. This assignment presents a Bash-based solution for the three administrative tasks and analyzes FCFS, SSTF, SCAN, and C-SCAN disk scheduling.

1. Command-Line Script and Challenges

The following Bash script automates daily backups, moves older log files into an archive directory, and checks disk usage on multiple servers.

```
#!/usr/bin/env bash

set -euo pipefail

BACKUP_DIR="/backup/bank"
LOG_SOURCE="/var/log/bank"
LOG_ARCHIVE="/var/log/bank/archive"
SOURCE_DATA="/srv/bank/data"

SERVERS=("bank-app-01" "bank-db-01" "bank-web-01")
THRESHOLD=80

DATE=$(date +"%Y-%m-%d")
BACKUP_FILE="$BACKUP_DIR/bank_data_$DATE.tar.gz"

mkdir -p "$BACKUP_DIR" "$LOG_ARCHIVE"

echo "=== Daily Bank Backup ==="

if tar -czf "$BACKUP_FILE" "$SOURCE_DATA"; then
   echo "Backup completed: $BACKUP_FILE"
else
   echo "Backup failed." >&2
   exit 1
fi

echo
echo "=== Moving Older Log Files ==="

find "$LOG_SOURCE" -maxdepth 1 -type f -name "*.log" \
   -mtime +1 -exec mv -t "$LOG_ARCHIVE" -- {} +

echo "Older log files moved to $LOG_ARCHIVE"

echo
echo "=== Monitoring Server Disk Usage ==="

for SERVER in "${SERVERS[@]}"; do
   echo "Checking $SERVER..."

   USAGE=$(ssh "$SERVER" "df -P / | awk 'NR==2 {gsub(/%/,\"\"); print \$5}'")

   if [ "$USAGE" -ge "$THRESHOLD" ]; then
      echo "WARNING: $SERVER disk usage is ${USAGE}%"
   else
```

```
        echo "$SERVER disk usage is ${USAGE}%"
    fi
done

echo
echo "All tasks completed."
```

The script uses tar to create a dated compressed backup, find and mv to move log files older than one day, and df through SSH to monitor disk usage on multiple servers. GNU documentation identifies df as a utility for reporting filesystem space usage, making it appropriate for the monitoring requirement (GNU Project, 2026). SSH also allows commands to be executed on remote machines, which makes centralized monitoring possible (OpenSSH, 2026).

Several challenges must be considered. First, the script needs appropriate permissions to access banking data and log directories. Second, backup failures must be detected rather than silently ignored. The set -euo pipefail configuration and conditional backup check help expose failures. Third, remote servers may be unavailable, so SSH errors should be logged and monitored in a production implementation. Fourth, active log files should not be moved while applications are writing to them; therefore, the example moves files older than one day rather than indiscriminately moving every .log file. Finally, backups should be stored on separate protected storage and tested through restoration procedures rather than assuming that a successfully created archive is automatically a valid recovery copy.

Bash scripts are suitable for this type of automation because a shell can combine operating-system utilities and execute commands non-interactively from a script (GNU Project, 2025). The script could also be scheduled using cron, which allows commands or programs to be executed at specified times (Vixie/ISC cron documentation, n.d.). For example, a daily execution could be scheduled with:

```
0 2 * * * /usr/local/bin/bank_maintenance.sh
```

This would run the maintenance script every day at 2:00 a.m.

2. FCFS and Alternative Disk Scheduling Algorithms

The bank currently uses First-Come, First-Served (FCFS) disk scheduling. FCFS services disk requests in the same order in which they arrive. Its primary advantage is simplicity and fairness based on arrival order. However, it does not consider the physical location of requests on an HDD. Consequently, the disk head may move back and forth unnecessarily between distant tracks, increasing seek time and reducing performance.

Arpaci-Dusseau and Arpaci-Dusseau (2018) explain that disk scheduling attempts to reduce the mechanical costs associated with disk-head movement. Their discussion also highlights an important weakness of SSTF: when requests continue to arrive near the current head position, requests farther away can experience starvation. SCAN addresses this problem by moving the disk head in a sweeping direction and servicing requests along the way. C-SCAN similarly sweeps in one direction but provides more uniform waiting behavior across disk locations.

Comparison of Scheduling Algorithms

Algorithm	Main principle	Advantage	Disadvantage
FCFS	Serves requests in arrival order	Simple and predictable	Can cause excessive head movement
SSTF	Serves the closest request	Usually reduces immediate seek distance	Can cause starvation
SCAN	Moves back and forth like an elevator	Good balance of performance and fairness	Some unnecessary movement to disk ends
C-SCAN	Moves in one direction and returns to the beginning	More uniform waiting time	Return movement adds overhead

SSTF could improve performance over FCFS because it normally selects the request requiring the shortest immediate seek. However, the algorithm can repeatedly favor requests close to the current head position, causing distant requests to wait for a long time. This is undesirable for a banking system because transaction-related storage requests should not be exposed to potentially unpredictable waiting times.

C-SCAN provides more uniform service because the disk head services requests in one direction before returning to the beginning. This can be useful when fairness and predictable waiting times are important. However, the return movement does not directly service requests.

Recommended Algorithm: SCAN

For the stated HDD-based scenario, I recommend SCAN because it provides a practical balance between seek efficiency and fairness. Instead of servicing requests blindly in arrival order like FCFS, SCAN moves systematically across the disk, serving pending requests during each sweep. This reduces the random back-and-forth movement that can occur with FCFS while avoiding the starvation problem associated with SSTF. OSTEP describes SCAN as the "elevator" algorithm because the head sweeps across the disk while servicing requests in its path.

For a bank processing a large number of daily transactions, this balance is important. FCFS may produce unnecessary seek movements, while SSTF can favor requests in one region of the disk. SCAN provides a more controlled service pattern and therefore represents a reasonable choice when the storage device is a traditional HDD.

It is important to note that modern storage systems may use device specific I/O schedulers rather than textbook FCFS, SSTF, or SCAN. For example, Linux supports schedulers such as mq-deadline, bfq, kyber, and none for block devices. Therefore, a production banking environment should benchmark its actual storage hardware and operating-system configuration before changing the scheduler.

Conclusion

Command-line scripting can significantly reduce repetitive administrative work in a banking environment. The proposed Bash script automates daily backups, log-file movement, and disk-usage monitoring across multiple servers while incorporating basic error handling and threshold-based warnings. For the storage problem, FCFS is simple but can generate unnecessary disk-head movement. SSTF can reduce immediate seek distance but may

cause starvation, while C-SCAN improves fairness but introduces additional return movement. For the HDD scenario described, SCAN provides a useful balance between efficient head movement and predictable service. In a real banking infrastructure, however, the final configuration should be validated through workload testing and monitoring the actual storage hardware.

References

Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018). *Operating systems: Three easy pieces*. Arpaci-Dusseau Books. https://pages.cs.wisc.edu/~remzi/OSTEP/file-disks.pdf

GNU Project. (2025). *Bash reference manual*. Free Software Foundation. https://www.gnu.org/software/bash/manual/

GNU Project. (2026). *GNU Coreutils manual*. Free Software Foundation. https://www.gnu.org/software/coreutils/manual/

OpenSSH. (2026). *ssh(1): OpenSSH remote login client*. Linux manual pages. https://man7.org/linux/man-pages/man1/ssh.1.html

OpenSSH. (2026). *rsync(1): A fast, versatile, remote and local file-copying tool*. Linux manual pages. https://man7.org/linux/man-pages/man1/rsync.1.html

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley. https://www.os-book.com/OS10/

The Linux Kernel Organization. (2023). *Switching I/O schedulers*. https://www.kernel.org/doc/html/v6.2/block/switching-sched.html